UNIVERSIDADE FEDERAL DO CEARÁ
Campus de Quixadá
Prof. Thiago Werlley
QXD0240- Robótica

Prática
2025.2

PRÁTICA 2 – Planejamento de Trajetória PRM e PF

Objetivo

Aprender a criar um planejador de caminhos por Campos Potenciais (*Potential Field*).
Aprender a criar um planejador de caminhos por Mapa de Rotas Probabilístico (*Probabilistic Roadmap*).

Parte 1 - Planejamento de Caminhos por Mapa de Rotas Probabilístico

1. Inicie o MatLab
2. No MatLab:

   a. Digite:

   *image = imread('**mapaexemplo.png**');*
   grayimage = rgb2gray(image);
   bwimage = grayimage < 0.5;
   grid = robotics.BinaryOccupancyGrid(bwimage, resolução);
   show(grid)

   Onde **mapaexemplo.png** é o nome do arquivo que você salvou na prática passada e resolução é um número inteiro que dividirá o tamanho do seu mapa para ter a resolução em metros desejada. **Ex.: imagem com 450x 360 pixels, com resolução 50**.
3. O robô é um ponto/pixel no mapa. Logo, defina o raio do robô conforme resolução do seu mapa. **Ex.: o Turtlebot geralmente tem** 0.2 m **de raio**. Digite então:

   robotRadius = 0.2;
   mapInflated = copy (grid);
   inflate(mapInflated, robotRadius);
   show(mapInflated)

   Isso irá inflar o mapa com respeito ao raio do robô.
4. Nessa primeira prática, vamos usar o algoritmo do PRM do próprio MatLab. Para isso precisamos definir os parâmetros do algoritmo tais como:

   a. Mapa
   prm = robotics.PRM;
   prm.Map = mapInflated;

   b. Local de partida do robô

startLocation = [1 0.5];

c. Local alvo
endLocation = [8 6];

d. Colocar o restante dos parâmetros para incluir os pontos inicial e final nos nós

prm.NumNodes = 20;
prm.ConnectionDistance = 15;
show(prm)

5. Agora vamos encontrar o caminho usando um algoritmo de busca. Nesse caso o MatLab usa o algoritmo de D

path = findpath(prm, startLocation, endLocation);
path

6. Note que o caminho retornado é vazio. Notou que no mapa gerado o número de nós foi pouco? Como resolver? Achando o número mínimo de nós que encontre um caminho.

while isempty(path)
% Se nenhum caminho for encontrado, aumente o número de nós prm.NumNodes = prm.NumNodes + 10;
% Use a função |update| para recriar o mapa de rotas PRM com
% o atributo modificado update(prm);
% Procure por um caminho com o PRM atualizado
path = findpath(prm, startLocation, endLocation);
end
%mostre o caminho final
path

7. Mostre agora o caminho dentro do mapa

show(prm)

8. Qual o número de nós resultante da busca?

prm.NumNodes

Parte 2 - Planejamento de Caminhos por Campos Potenciais

Vamos agora usar o mesmo mapa e aplicar os campos potenciais para que possamos distorcer esse mapa através dos campos. Nessa prática vamos ficar por aqui e deixar a função de navegação como tarefa de casa. Para distorcerem o mapa, vamos começar primeiramente repetindo os 3 passos anteriores.

1. Inicie o MatLab
2. No MatLab:

a. Digite:

```
resolucao = 50;
imagem = imread('mapaexemplo.bmp');
grayimg = rgb2gray(imagem);
binImage = grayimg < 0.5;
grid = robotics.BinaryOccupancyGrid(binImage,
   resolucao);
show(grid)
figure

startLoc = [1.83  5.01];
endLoc = [8.35  5.45];
```

Onde **mapaexemplo.png** é o nome do arquivo que você salvou na prática passada e resolução é um número inteiro que dividirá o tamanho do seu mapa para ter a resolução em metros desejada. **Ex.: imagem com 450x 360 pixels, com resolução 50**.

3. O robô é um ponto/pixel no mapa. Logo, defina o raio do robô conforme resolução do seu mapa. **Ex.: o Turtlebot geralmente tem** 0.2 m **de raio**. Digite então:

```
robotRadius = 0.177;
mapInflated = copy(grid);
inflate(mapInflated,robotRadius);
```

Isso irá inflar o mapa com respeito ao raio do robô.

4. Agora começaremos a distorcer o mapa gerado com respeito ao campo potencial. Primeiramente temos que transformar o mapa que está em Occupancy Grid para uma matriz e plotar ela como superfície para que possamos entender o efeito do campo potencial. Para tal, digite:

matriz = occupancyMatrix(mapInflated);
Watr=15;
*matrix=flip(matriz*Watr);*
surface(matrix)

Nesse caso, Wo é o peso que daremos ao obstáculo. É um parâmetro configurável.

5. Para distorcer o mapa temos que criar uma malha de distorção chamada de Mesh Grid, ou malha em grade. Para isso criamos a malha com o mesmo tamanho do nosso mapa. No meu caso, tenho um mapa de 360x450 pixels. Logo, digito:

xf = 0:0.01995555556:8.98; %450
yf = 0:0.01994444444:7.18-0.01994444444; %360
[X,Y] = meshgrid(xf,yf);
Nesse caso, o valor de 12 é o peso da malha (inclinação por assim dizer) e os números do meio são a resolução para que, no final, *xf* tenha um tamanho de 450 e *yf* de 360.

6. Temos também que dizer qual o ponto (em metros) que nosso alvo tem que estar. Como meu mapa tem 9x12 eu olho na figura 2 gerada no passo 3 e escolho um alvo. Nesse caso eu digito:
   startLoc = [1.83 5.01];
   endLoc = [8.35 5.45];
7. Por fim, atribui um peso (KG) para o mínimo global do Alvo, crio uma função de atração (distorção do mapa) com respeito ao alvo multiplico isso ao peso dado e crio um mapa final digitando:
   KA=15;
   aux=sqrt((endLoc(1)-X).^2+((endLoc(2)-Y).^2));
   *VG=KA*aux;*
   Temp=VG+matrix;
   figure
   surface(xf,yf,Temp)
8. Criando os obstáculos

```
[ox, oy]=size(matrix);
count=1;
for y=1:7:ox
  for x=1:7:oy
    if matrix(y,x)==15
      obstacles{count}=[x*0.01995555556, y*0.01994444444,
         robotRadius];
      plot(obstacles{count}(1),obstacles{count}(2), 'o')
      hold on
      count=count+1;
    end
  end
end
count=count-1;
```

9. Resumindo os Plots

```
atual{1}=startLoc;
cntpose = 1;
ka=0.1;
kr=0.002;
DO=0.2;
dist=1;
gradX=0;
gradY=0;
figure
show(mapInflated);
hold on
plot(endLoc(1), endLoc(2), 'bo');
hold on
plot(startLoc(1), startLoc(2), 'bx');
hold on
for i=1:1:count
```

```
  plot(obstacles{count}(1),obstacles{count}(2), 'ro')
  hold on
end
```

10. Gerando a trajetória

```
while (dist>=0.01)
  for m=1:1:count
    RobotObstacleDistance =
       sqrt((atual{cntpose}(1)-obstacles{m}(1))^2+
    (atual{cntpose}(2)-obstacles{m}(2))^2)-obstacles{m}(3);
    if RobotObstacleDistance <= DO
      gradX = gradX +
         kr*(atual{cntpose}(1)-obstacles{m}(1));
      gradY = gradY +
         kr*(atual{cntpose}(2)-obstacles{m}(2));
    end
  end

  dist=sqrt((endLoc(1)-atual{cntpose}(1))^2+(endLoc(2)-
  atual{cntpose}(2))^2);
  derivX = ka*(endLoc(1)-atual{cntpose}(1))/dist;
  derivY = ka*(endLoc(2)-atual{cntpose}(2))/dist;

  atual{cntpose+1}(1)=atual{cntpose}(1)+(derivX+gradX);
  atual{cntpose+1}(2)=atual{cntpose}(2)+(derivY+gradY);

  plot(atual{cntpose}(1),atual{cntpose}(2),'go');
  hold on

  if dist<=0.1
    break;
  end
  cntpose=cntpose+1;
end
```

11. Mostre ao professor o mapa resultante.

Ao final da prática:

1. Demonstrar in loco o funcionamento da configuração solicitada mostrando os resultados ao professor (ou monitor);
2. Preparar um relatório segundo o modelo que se encontra no Sigaa contendo a seguintes explicações:
   a. Explique e demonstre os resultados da primeira parte.
   b. Explique e demonstre os resultados da segunda parte.
   c. Compara os resultados da primeira com a segunda parte explicando fraquezas e vantagens de ambas.